

Approximate Nearest Neighbor Search for Robot Trajectory Caching: Final Report

Valmiki Kothare

May 2026

1 Motivation

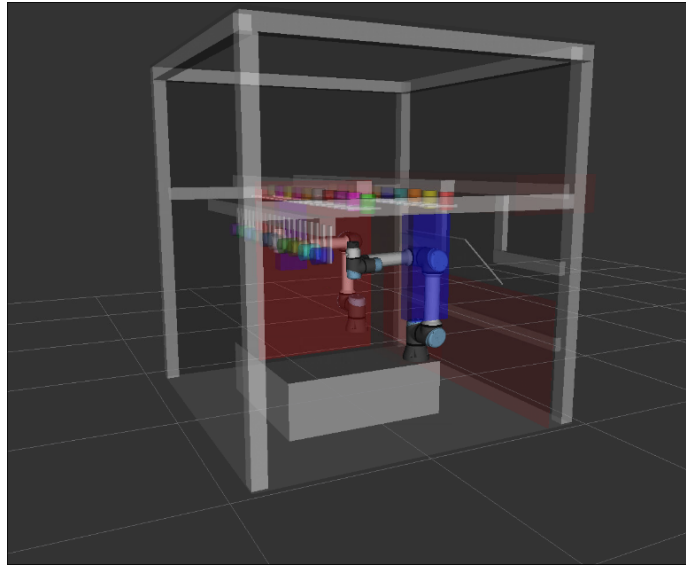


Figure 1: Simulated 6-DOF robot in a tabletop environment used for all benchmarks.

In my work with 6-DOF robots, motion planning becomes prohibitively computationally expensive as more collision objects are added to the environment. In many robotics automation scenarios, however, the robot operates in a largely static environment, and the same start configurations and goal constraints are repeatedly encountered—both within a single program execution and across multiple runs. If we can build up a cache of previously computed trajectories

and query it at runtime with the current plan request, we can dramatically reduce planning time. For this to be practical the cache must satisfy three requirements:

- Updating and querying the cache must be significantly more computationally efficient than planning from scratch.
- Cache misses must be minimized by retrieving trajectories *close enough* to the start and goal of the query.
- The cache size must be bounded by storing only the best trajectories for each query.

These requirements make trajectory caching a natural instance of the Approximate Nearest Neighbor (ANN) Search problem: given a plan request (start state, goal state or pose), find the cached trajectories whose stored request lies within a per-coordinate tolerance of the query.

2 Approach

All four cache backends implement the same Mapping-like API over `PlanRequest` $\rightarrow$ `RobotTrajectory`. Each plan request is reduced to a feature vector of joint angles (and optionally a Cartesian pose), and a backend reports a hit when any stored fingerprint lies within configured per-coordinate tolerances of the query. The backends differ only in how they index this feature space. Using standard ANN notation: N = entries in the cache, B = candidate trajectories per match group, d = feature-vector dimensionality ($6 + 6 = 12$ for joint-space goals, $6 + 7 = 13$ for Cartesian goals).

2.1 Linear brute-force

Each request fingerprint is stored verbatim with no quantization. On query, every stored fingerprint is scanned and the per-coordinate check $|x_i - y_i| \leq \tau_i$ is applied. **Query:** $O(N \cdot d)$ full scan. **Update:** $O(1)$ amortized insertion. Recall is exactly the configured tolerance with no aliasing, but query cost grows linearly with cache size—this is the brute-force baseline that an ANN structure should improve upon.

2.2 Spatially-binned hashing

Each float coordinate of the request fingerprint is quantized via $\lfloor x/\tau \rfloor$ to produce a deterministic integer-bin key. Each bin holds up to B candidates sorted by path cost, and the bin map is maintained in memory and serialized on close. **Query:** $O(1)$ hash lookup. **Update:** $O(\log B)$ bisect-insert with no per-operation disk I/O during the session. The key caveat is *bin-boundary aliasing*: coordinates near $k\tau$ toggle into adjacent bins under tiny floating-point drift or physical robot inaccuracy, so the same physical request can produce different keys across calls.

2.3 Spatially-binned hashing (LMDB-backed)

Identical quantization strategy to the in-memory variant, but the bin map is stored on disk in an embedded database. Each bin holds up to B candidates sorted by path cost. **Query:** $O(1)$ hash lookup plus a read transaction. **Update:** $O(\log B)$ bisect-insert plus one write transaction committed to disk per call—the cost of crash-durable persistence. Shares the same bin-boundary aliasing weakness.

2.4 K-d tree

Each request is mapped to a d -dimensional feature vector whose coordinates are pre-scaled by $1/\tau$ so that the per-coordinate tolerance becomes a unit-radius L^∞ ball. Separate trees are maintained per goal type and queried with an L^∞ -ball radius-1 lookup. **Query:** $O(\log N + k)$ average case, with a one-time $O(N \log N)$ rebuild triggered lazily on the first query after any insert. **Update:** $O(1)$ amortized—the tree rebuild is deferred. Preserves exact per-coordinate tolerance semantics with no aliasing.

3 Experimental Setup

Two benchmarks were run on a 6-DOF robot in a static tabletop environment.

Planning benchmark. For each backend: (1) a *collect phase* in which 25 random goals were sampled around an idle configuration and planned end-to-end using MoveIt, with each executed trajectory stored in the cache (including the reverse leg); (2) a *cycle phase* in which the same 25 goals were replayed five times. Per leg we recorded plan/query time, execution time, cache hit status (`from_cache`), and current cache size.

Direct I/O benchmark. To observe asymptotic behavior, the plan/execute layer was stripped out entirely. On a cache miss, a two-waypoint dummy trajectory was synthesized and inserted via the same `cache_trajectory(...)` path used in production. This allows the cache to grow orders of magnitude faster than the planning benchmark permits, making the complexity class of each backend directly observable.

4 Results: Planning Benchmark

4.1 Cache hit rate

Brute-force and k-d tree both land near the 50% collect-phase ceiling (47% and 46% respectively)—reverse-trajectory caching stores the return leg alongside the outbound one, so roughly half the collect-phase legs are expected to hit

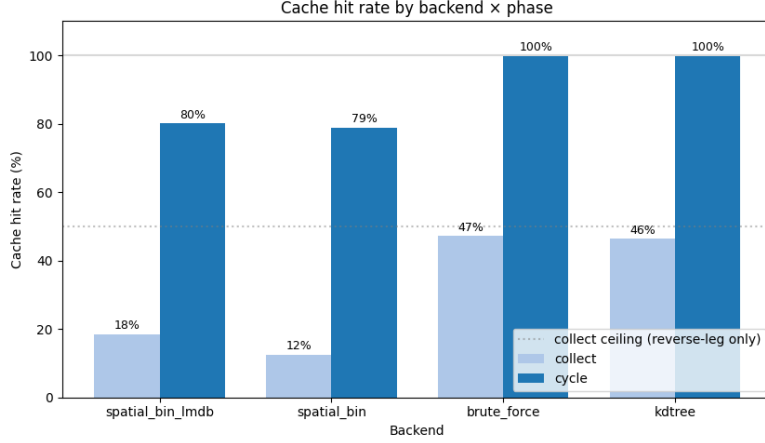


Figure 2: Cache hit rate by backend and phase. The dashed line at 50% marks the theoretical ceiling for the collect phase (only the reverse leg can hit).

(Figure 2). Both backends sustain approximately 100% hit rate through all five replay cycles.

The spatially-binned hashing backends fall well below the ceiling in the collect phase (dict-based: 13%, LMDB-backed: 19%) and never reach 100% in the cycle phase—climbing from 61%/69% in cycle 1 to only 90%/86% by cycle 5. The shortfall stems from bin-boundary aliasing at the arm’s idle position, where several joints sit at exactly 0 rad. The quantization $\lfloor 0/\tau \rfloor$ is numerically unstable: tiny floating-point drift across zero flips the bin key between 0 and -1 , making a stored trajectory invisible to a query that hashes to the other key—a persistent miss regardless of how many times that goal has been cached.

4.2 Plan/query time: hits vs. misses

With the fast planner used in this benchmark, cache hits are not substantially cheaper than fresh plans—hit and miss time distributions overlap for every backend (Figure 3). For the spatially-binned hashing and k-d tree backends, median hit time is actually slightly higher than miss time (~ 0.5 – 1 ms extra), reflecting cache deserialization and validation overhead that is comparable to the planner’s own latency. Brute-force’s $O(N \cdot d)$ scan runs on every call regardless of outcome, so its hit and miss distributions are similarly narrow. The fast planner is intentional—it keeps total benchmark runtime manageable—but in a real workload with complex plans (often seconds each) the speedup from a cache hit would dominate lookup overhead.

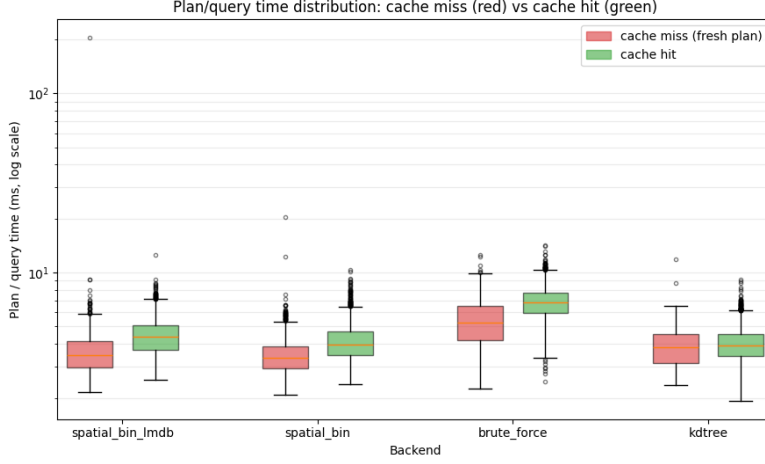


Figure 3: Plan/query time distribution (log scale) for cache hits (green) and cache misses (red), grouped by backend.

4.3 Cumulative planning time

During the collect phase, all four backends accumulate planning time at roughly the same rate (Figure 4)—most legs miss and pay the full OMPL budget. After the phase boundary the slopes diverge. Brute-force carries the steepest slope into the cycle phase despite its $\sim 100\%$ hit rate: its $O(N \cdot d)$ full scan runs on every `plan()` call (hit or miss), adding ~ 7 ms per leg and making it the slowest backend per cycle (~ 4.1 s for 300 legs). The k-d tree and dict-based spatially-binned backend are fastest (~ 2.4 s per cycle); the LMDB-backed variant sits slightly higher (~ 2.6 s) due to read-transaction overhead on each hit. The spatially-binned hashing backends also continue accumulating cost through replay cycles because 10–40% of their legs still miss and trigger full OMPL plans—a direct consequence of bin-boundary aliasing.

4.4 Hit rate across replay cycles

Brute-force and k-d tree saturate at $\sim 100\%$ from cycle 1 and hold there (Figure 5)—exact per-coordinate tolerance matching finds stored trajectories reliably on every replay. The spatially-binned hashing backends never converge: starting at 61% and 69% respectively, they improve steadily across cycles (reaching 90% and 86% by cycle 5), but cannot saturate. Bin-boundary aliasing creates a self-reinforcing dynamic: each missed leg is planned fresh and re-inserted under the new aliased bin key, gradually broadening coverage—but joints permanently near 0 rad continue to hash inconsistently, so some fraction of goals remain unreachable by the cache regardless of how many times they have been stored.

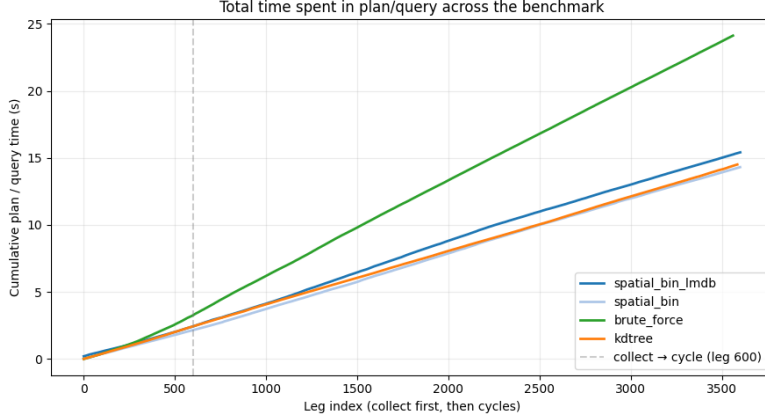


Figure 4: Cumulative plan/query time across all legs. The dashed vertical line marks the collect-to-cycle phase boundary.

4.5 Query time vs. cache size

At the scale of the planning benchmark, query time is essentially flat for all backends (Figure 6). Brute-force shows a modestly positive regression slope consistent with its $O(N \cdot d)$ scan; the hash-based backends hold near-zero slope confirming $O(1)$ lookup. The aliasing story is also visible in the cache sizes themselves: brute-force and k-d tree stabilise near 1 292–1 308 entries (consistent with ~ 300 goals $\times$ 4 bins), while the spatially-binned hashing backends reach 3 728–4 010 entries (roughly $3\times$ the baseline) and continue growing through all five replay cycles, because each aliased miss inserts a fresh bin without adding any reachable trajectory.

5 Results: Direct I/O Benchmark

The planning benchmark coupled every cache insertion with a full MoveIt plan and execute round-trip, limiting the cache to a few hundred entries per backend. The direct benchmark strips out the plan/execute layer, synthesizing dummy trajectories on every miss and inserting them immediately. This allows the cache to grow to $\sim 40\,000$ entries per backend, making the asymptotic complexity class of each backend directly observable.

5.1 Query time vs. cache size (direct)

At $\sim 40\,000$ entries the asymptotic complexity classes become clearly visible (Figure 7). The spatially-binned hashing backends hold flat ($O(1)$), with the dict-based variant at a lower median (~ 0.14 ms) than the LMDB-backed variant (~ 0.31 ms) due to read-transaction overhead on each lookup. Brute-force’s median climbs with a power exponent of $\approx +1.02$, consistent with $O(N \cdot d)$: at peak

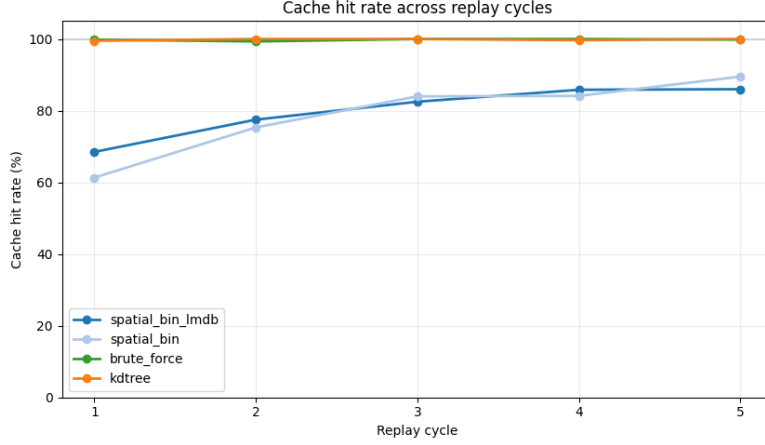


Figure 5: Cache hit rate for each replay cycle in the cycle phase.

cache size it reaches ~ 78 ms—over $530\times$ slower than the dict-based spatially-binned backend. The k-d tree shows a power exponent of ≈ -0.8 , reflecting sub-linear amortized scaling: the $O(N \log N)$ rebuild, paid once per insert, is amortized across all subsequent queries between rebuilds, so per-query cost decreases as N grows, with an overall median of ~ 0.41 ms, falling to ~ 0.32 ms at peak size as rebuilds are amortized over more queries.

5.2 Update time vs. cache size (direct)

Insert cost is near-constant for every backend (power exponents all < 0.05), confirming that the cache’s amortized write cost does not scale with N (Figure 8). The backends differ significantly in absolute insert latency: the dict-based spatially-binned backend and k-d tree are fastest (~ 1.0 ms); brute-force is slightly slower (~ 1.3 ms) due to full trajectory serialization. The LMDB-backed spatially-binned backend is an order of magnitude slower (~ 14 ms per insert) because each write commits a disk transaction—the price of crash-durable persistence.

5.3 Cumulative throughput (direct)

Cumulative query time fans out sharply between backends as cache size grows: brute-force’s slope steepens visibly with each query while the spatially-binned hashing backends remain on a straight line (Figure 9). Cumulative update time stays roughly parallel across backends because every backend’s insert is approximately constant-cost per call. The total time budget is therefore dominated by the *query* path at large N for brute-force, but by the *update* path for the LMDB-backed spatially-binned backend regardless of size.

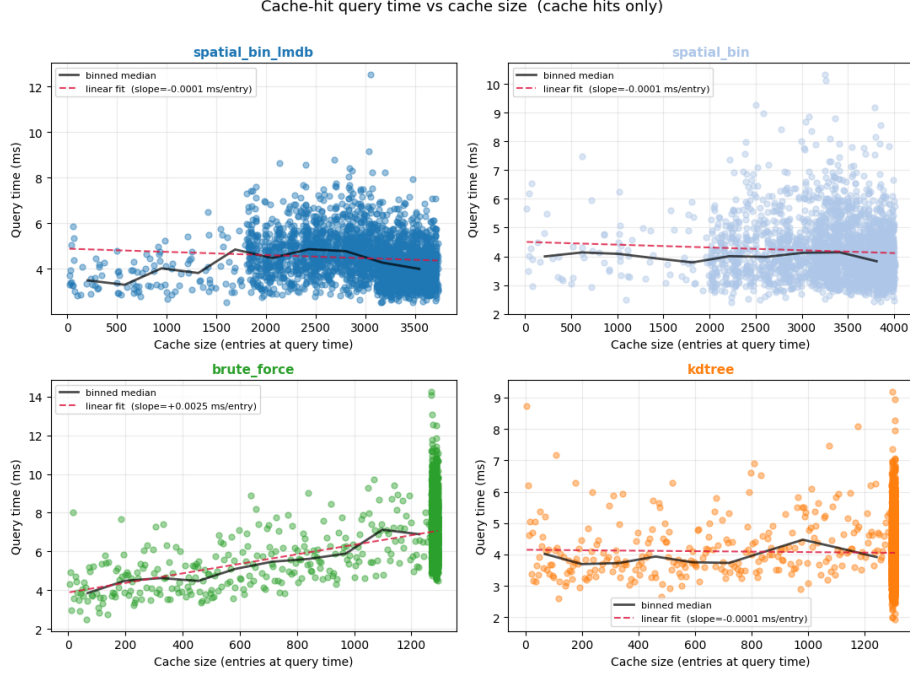


Figure 6: Cache-hit query time vs. cache size at query time (one panel per backend). Red dashed lines are linear regression fits; black lines are binned medians.

6 Conclusions

1. **Exact-tolerance backends achieve near-perfect recall; spatially-binned hashing backends do not.** Brute-force and k-d tree sustain $\sim 100\%$ hit rate across all five replay cycles. The spatially-binned hashing backends top out at 86–90% by cycle 5 and never converge.
2. **Bin-boundary aliasing is the root cause, and it inflates cache size by $\sim 3\times$.** The arm’s idle position holds several joints at exactly 0 rad. Quantization $[0/\tau]$ is numerically unstable: floating-point drift across zero flips the bin key, making stored trajectories invisible to queries that hash to a different key. Aliased misses insert new bins without adding reachable trajectories, growing the spatially-binned hashing backends to ~ 4000 entries versus ~ 1300 for the exact-tolerance backends—and the index continues growing through replay cycles.
3. **With the fast planner used here, cache hits are not meaningfully faster than misses.** Cache lookup overhead ($\sim 4\text{--}7$ ms) is comparable to the planner’s own latency. In production with complex plans (seconds each) the speedup from a cache hit would be substantial. Brute-force’s

Direct benchmark — query time vs cache size (collect phase only)

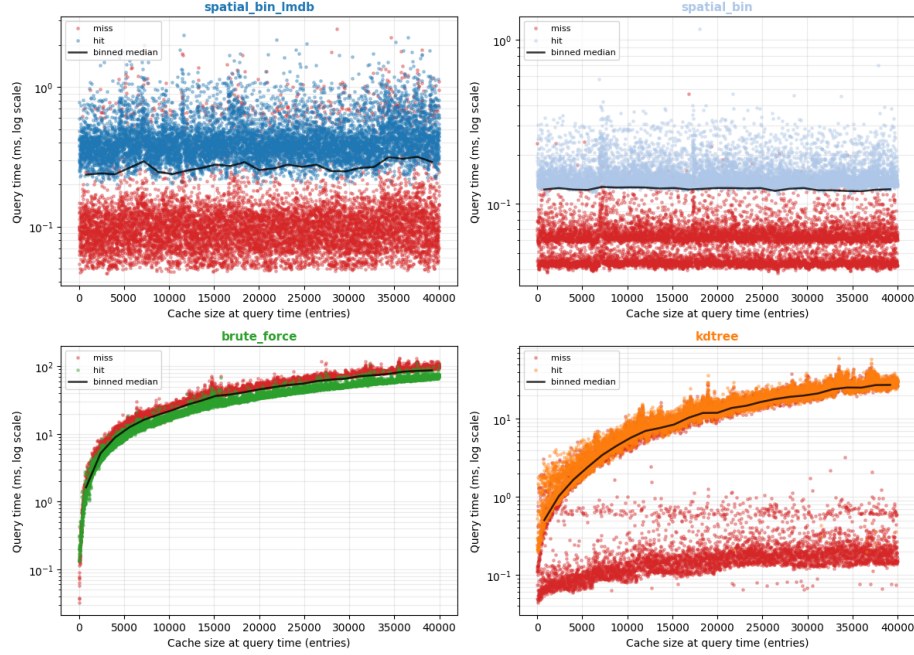


Figure 7: Direct benchmark: query time vs. cache size (collect phase only, log-y scale).

$O(N \cdot d)$ scan runs on every call, making it the highest-latency backend per cycle (~ 4.1 s) despite its near-perfect hit rate.

4. **Query-time scaling confirms the theoretical complexity classes.** Hash-based backends (both spatially-binned hashing variants) show $O(1)$ query time; brute-force’s query time grows as $O(N \cdot d)$ (power exponent $\approx +1.02$ in the direct benchmark, reaching ~ 78 ms at 40 000 entries); k-d tree shows sub-linear amortized scaling (exponent ≈ -0.8) due to rebuild amortization, with an overall median of ~ 0.41 ms (dropping to ~ 0.32 ms at peak size).
5. **K-d tree is the best overall choice for this workload.** It matches brute-force’s exact per-coordinate recall while keeping amortized query time sub-linear in cache size—confirmed at ~ 40 000 entries in the direct benchmark. If crash-durable persistence is required without sacrificing $O(1)$ query time, the LMDB-backed spatially-binned hashing backend is viable—but its ~ 14 ms insert cost and irrecoverable 10–14% miss rate should be weighed against the target workload.

Assisted by AI for code generation, analysis, and writing

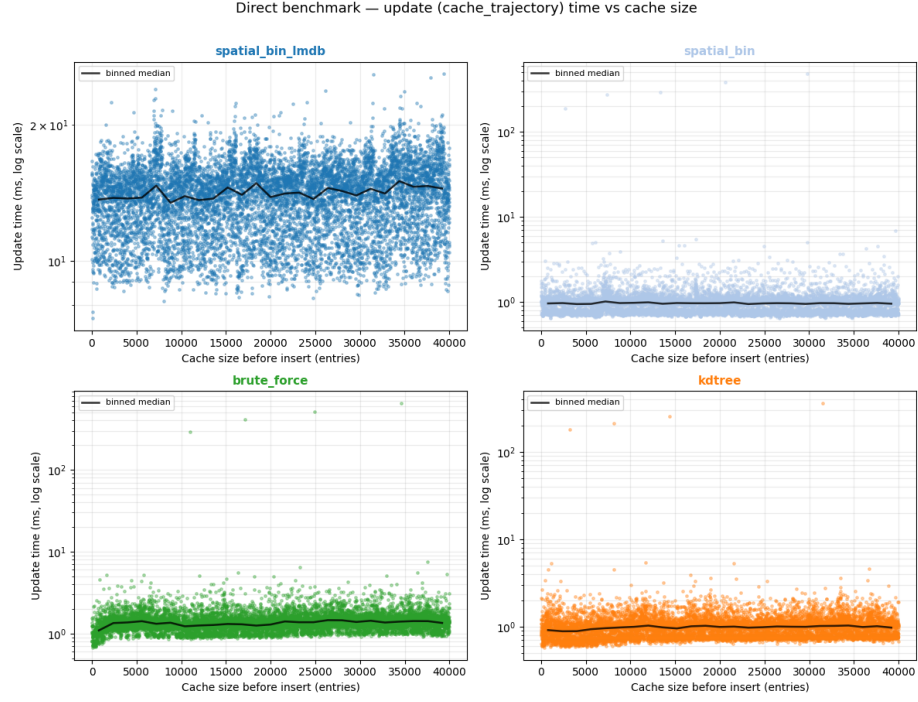


Figure 8: Direct benchmark: update (insert) time vs. cache size, misses only (log-y scale).

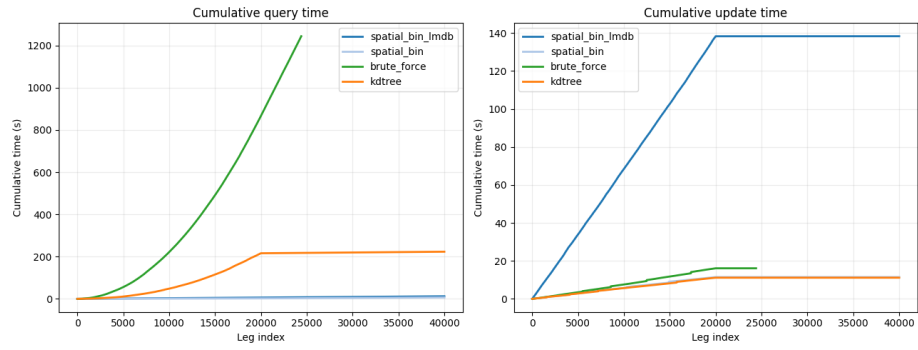


Figure 9: Direct benchmark: cumulative query time (left) and update time (right) as the benchmark progresses.